

BLACK FRIDAY SALES ANALYSIS

Project objective

- Who are the customers?
- Which demographic groups purchase more?
- Which product categories generate more revenue?
- Which customers are high-value customers?
- Does gender affect purchasing behavior?
- Does marital status affect purchasing behavior?
- Which cities contribute more to sales?
- What products are commonly purchased together?
- What business strategies can be recommended?

ABOUT DATASET:

Dataset: Black Friday Sales Analysis

- Number of Rows : 550068
- Number of Columns : 12
- Columns Include : User_ID,Product_ID,Gender,Age,Occupation,City_Category, Stay_In_Current_City_Years, Marital_Status,Product_Category_1,Product_Category_2, Product_Category_3 and Purchase.

Data Overview

The first step is to get a quick overview of the dataset. This gives an initial understanding of the dataset's structure, including the number of rows, columns, data types, sample data, and basic statistics.

Actions:

- **Preview:** Displaying the first few rows of the data to get an idea of how it looks.
- **Shape:** Checking the number of rows and columns in the dataset.
- **Data types:** Understanding the data types for each column (e.g., numeric, categorical).
- **Summary Statistics:** Providing a statistical summary for the numerical features.

```
In [1]: # import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from scipy.stats import ttest_ind
from scipy import stats
```

Data Loading and Initial Overview

```
In [2]: # step:1 Load the dataset
df=pd.read_csv("Black Friday Data.csv")
```

```
In [3]: df.head()
```

Out[3]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_!
0	1000001	P00069042	F	0-17	10	A	2	
1	1000001	P00248942	F	0-17	10	A	2	
2	1000001	P00087842	F	0-17	10	A	2	
3	1000001	P00085442	F	0-17	10	A	2	
4	1000002	P00285442	M	55+	16	C	4+	

```
In [4]: # understand the dataset
df.shape
```

Out[4]: (550068, 12)

```
In [5]: # column names
df.columns
```

Out[5]: Index(['User_ID', 'Product_ID', 'Gender', 'Age', 'Occupation', 'City_Category', 'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category_1', 'Product_Category_2', 'Product_Category_3', 'Purchase'], dtype='object')

```
In [6]: # data types
df.dtypes
```

```
Out[6]: User_ID          int64
Product_ID         object
Gender             object
Age               object
Occupation         int64
City_Category      object
Stay_In_Current_City_Years  object
Marital_Status     int64
Product_Category_1 int64
Product_Category_2 float64
Product_Category_3 float64
Purchase           int64
dtype: object
```

```
In [7]: # dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   User_ID                               550068 non-null int64
1   Product_ID                            550068 non-null object
2   Gender                                550068 non-null object
3   Age                                    550068 non-null object
4   Occupation                            550068 non-null int64
5   City_Category                         550068 non-null object
6   Stay_In_Current_City_Years            550068 non-null object
7   Marital_Status                        550068 non-null int64
8   Product_Category_1                    550068 non-null int64
9   Product_Category_2                    376430 non-null float64
10  Product_Category_3                    166821 non-null float64
11  Purchase                              550068 non-null int64
dtypes: float64(2), int64(5), object(5)
memory usage: 50.4+ MB
```

```
In [8]: # Statistical summary
df.describe()
```

```
Out[8]:
```

	User_ID	Occupation	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3
count	5.500680e+05	550068.000000	550068.000000	550068.000000	376430.000000	166821.000000
mean	1.003029e+06	8.076707	0.409653	5.404270	9.842329	10.000000
std	1.727592e+03	6.522660	0.491770	3.936211	5.086590	10.000000
min	1.000001e+06	0.000000	0.000000	1.000000	2.000000	1.000000
25%	1.001516e+06	2.000000	0.000000	1.000000	5.000000	1.000000
50%	1.003077e+06	7.000000	0.000000	5.000000	9.000000	1.000000
75%	1.004478e+06	14.000000	1.000000	8.000000	15.000000	1.000000
max	1.006040e+06	20.000000	1.000000	20.000000	18.000000	1.000000

```
In [9]: # Data Quality Check
# Missing values
df.isnull().sum()
```

```
Out[9]: User_ID          0
Product_ID          0
Gender              0
Age                0
Occupation          0
City_Category       0
Stay_In_Current_City_Years  0
Marital_Status      0
Product_Category_1   0
Product_Category_2  173638
Product_Category_3  383247
Purchase            0
dtype: int64
```

```
In [10]: # Also calculate percentage:
missing_percentage = df.isnull().mean() * 100

missing_percentage.sort_values(ascending=False)
```

```
Out[10]: Product_Category_3    69.672659
Product_Category_2           31.566643
User_ID                      0.000000
Product_ID                   0.000000
Gender                       0.000000
Age                          0.000000
Occupation                   0.000000
City_Category                 0.000000
Stay_In_Current_City_Years    0.000000
Marital_Status                0.000000
Product_Category_1            0.000000
Purchase                      0.000000
dtype: float64
```

```
In [11]: # Duplicate Check
df.duplicated().sum()
```

```
Out[11]: 0
```

```
In [12]: # Product Category 1
df["Product_Category_1"] = df["Product_Category_1"].astype(str)

# Product Category 2
df["Product_Category_2"] = (df["Product_Category_2"].fillna(0).astype(int).astype(str)

# Product Category 3
df["Product_Category_3"] = (df["Product_Category_3"].fillna(0).astype(int).astype(str)
```

Converted category columns to string as they are categorical variables. Replaced missing values in Category 2 & 3 with "Unknown" to preserve missing information.

Checking Unique Values

Checked the unique values in all categorical columns to understand the different categories present in the dataset and identify any unexpected or inconsistent values.

```
In [13]: # Check Unique Values
for col in df.select_dtypes(include="object"):
    print("\n", col)
    print(df[col].unique())

Product_ID
['P00069042' 'P00248942' 'P00087842' ... 'P00370293' 'P00371644'
 'P00370853']

Gender
['F' 'M']

Age
['0-17' '55+' '26-35' '46-50' '51-55' '36-45' '18-25']

City_Category
['A' 'C' 'B']

Stay_In_Current_City_Years
['2' '4+' '3' '1' '0']

Product_Category_1
['3' '1' '12' '8' '5' '4' '2' '6' '14' '11' '13' '15' '7' '16' '18' '10'
 '17' '9' '20' '19']

Product_Category_2
['Unknown' '6' '14' '2' '8' '15' '16' '11' '5' '3' '4' '12' '9' '10' '17'
 '13' '7' '18']

Product_Category_3
['Unknown' '14' '17' '5' '4' '16' '15' '8' '9' '13' '6' '12' '3' '18' '11'
 '10']
```

```
In [14]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   User_ID                               550068 non-null  int64
1   Product_ID                            550068 non-null  object
2   Gender                                550068 non-null  object
3   Age                                    550068 non-null  object
4   Occupation                             550068 non-null  int64
5   City_Category                          550068 non-null  object
6   Stay_In_Current_City_Years            550068 non-null  object
7   Marital_Status                         550068 non-null  int64
8   Product_Category_1                     550068 non-null  object
9   Product_Category_2                     550068 non-null  object
10  Product_Category_3                     550068 non-null  object
11  Purchase                               550068 non-null  int64
dtypes: int64(4), object(8)
memory usage: 50.4+ MB
```

Exploratory Data Analysis

Most purchases are around ₹8,047.

Most purchases are below the average ₹9,264.

A few high purchases increase the average.

```
In [15]: # Exploratory Data Analysis
numerical_cols = ['Purchase']
mean_values     = df[numerical_cols].mean()
median_values   = df[numerical_cols].median()
mode_values     = df[numerical_cols].mode().iloc[0]
range_values    = df[numerical_cols].max() - df[numerical_cols].min()
variance_values = df[numerical_cols].var()
std_dev_values  = df[numerical_cols].std()
skewness_values = df[numerical_cols].skew()
kurtosis_values = df[numerical_cols].kurt()

# Create Clean DataFrame
stats_df = pd.DataFrame({
    "Mean"           : mean_values,
    "Median"         : median_values,
    "Mode"           : mode_values,
    "Range"          : range_values,
    "Variance"       : variance_values,
    "Standard Deviation": std_dev_values,
    "Skewness"       : skewness_values,
    "Kurtosis"       : kurtosis_values
})

print(stats_df)
```

	Mean	Median	Mode	Range	Variance	Standard Deviation	\
Purchase	9263.968713	8047.0	7011	23949	2.523119e+07	5023.065394	

	Skewness	Kurtosis
Purchase	0.60014	-0.338378

Purchase Analysis

```
In [16]: df["Purchase"].describe()
```

```
Out[16]: count    550068.000000
mean      9263.968713
std       5023.065394
min       12.000000
25%      5823.000000
50%      8047.000000
75%     12054.000000
max     23961.000000
Name: Purchase, dtype: float64
```

```
In [17]: # Minimum purchase
df["Purchase"].min()
```

```
Out[17]: 12
```

```
In [18]: # Maximum purchase
df["Purchase"].max()
```

```
Out[18]: 23961
```

```
In [19]: # Average purchase
df["Purchase"].mean()
```

```
Out[19]: 9263.968712959126
```

```
In [20]: # Median purchase
df["Purchase"].median()
```

```
Out[20]: 8047.0
```

Outlier Analysis

```
In [21]: # Identify outliers
Q1 = df["Purchase"].quantile(0.25)
Q3 = df["Purchase"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
```

```
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
```

```
Q1: 5823.0
Q3: 12054.0
IQR: 6231.0
Lower Limit: -3523.5
Upper Limit: 21400.5
```

```
In [22]: # Identify outliers using the IQR limits already computed
outliers = df[(df["Purchase"] < lower_limit) | (df["Purchase"] > upper_limit)]

print(f"Number of outliers: {len(outliers)}")
print(f"Percentage of data: {len(outliers)/len(df)*100:.2f}%")

# Inspect the highest outliers
outliers[["User_ID", "Product_ID", "Purchase"]].sort_values(
    "Purchase",
    ascending=False
).head(10)
```

```
Number of outliers: 2677
Percentage of data: 0.49%
```

```
Out[22]:
```

	User_ID	Product_ID	Purchase
87440	1001474	P00052842	23961
93016	1002272	P00052842	23961
370891	1003160	P00052842	23961
292083	1003045	P00052842	23960
503697	1005596	P00117642	23960
321782	1001577	P00052842	23960
349658	1005848	P00119342	23960
437804	1001387	P00086242	23959
298378	1003947	P00116142	23959
7542	1001178	P00116142	23958

```
In [23]: plt.figure(figsize=(10, 5))

sns.histplot(df["Purchase"], bins=50, kde=True)

plt.axvline(
    upper_limit,
    linestyle="--",
    label=f"Upper Limit = {upper_limit:.0f}"
)

plt.axvline(
    Q1,
```

```

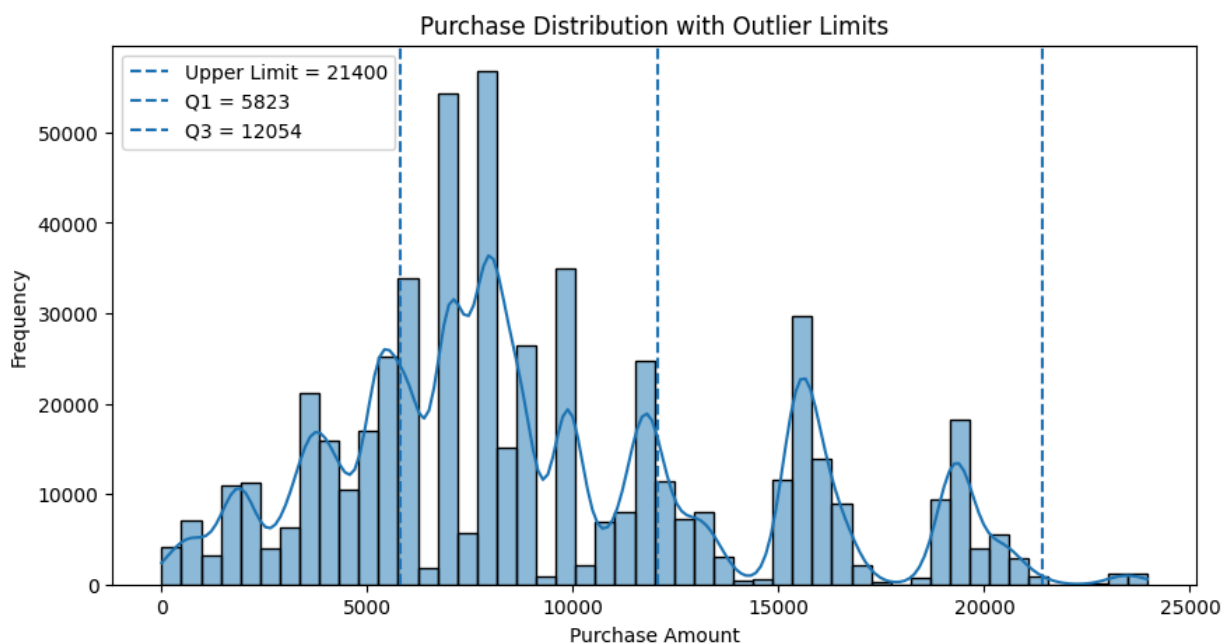
linestyle="--",
label=f"Q1 = {Q1:.0f}"
)

plt.axvline(
    Q3,
    linestyle="--",
    label=f"Q3 = {Q3:.0f}"
)

plt.title("Purchase Distribution with Outlier Limits")
plt.xlabel("Purchase Amount")
plt.ylabel("Frequency")
plt.legend()

plt.show()

```



Purpose: To identify the distribution of purchase amounts and detect potential outliers.

Histogram: Shows how purchase values are distributed.

Q1 & Q3: Show the middle 50% of purchase values.

Upper Limit: Helps identify unusually high purchases.

KDE line: Shows the overall shape of the distribution.

Conclusion: Most purchases are within the normal range, while values above 21,400 can be considered potential outliers.

```

In [24]: plt.figure(figsize=(10, 4))

sns.boxplot(x=df["Purchase"])

plt.axvline(
    upper_limit,

```

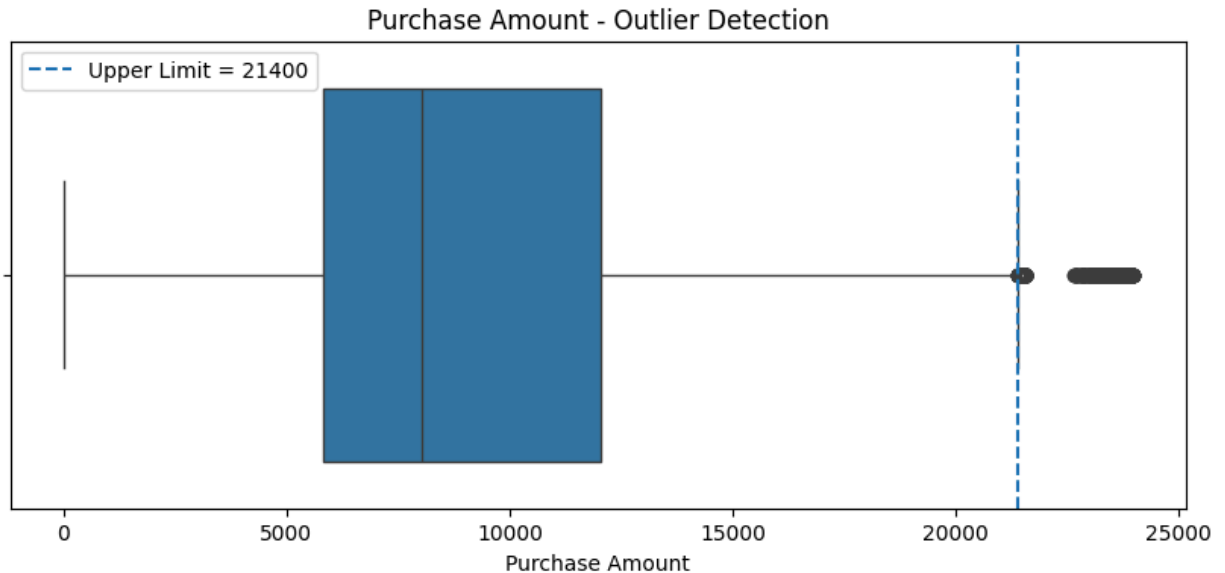
```

linestyle="--",
label=f"Upper Limit = {upper_limit:.0f}"
)

plt.title("Purchase Amount - Outlier Detection")
plt.xlabel("Purchase Amount")
plt.legend()

plt.show()

```



Purpose: To identify and visualize outliers in purchase amounts.

Boxplot: Shows the spread of purchase values.

Blue dashed line: Represents the upper limit of 21,400.

Dots after the limit: Represent potential outliers.

Conclusion: Most purchases are within the normal range, while purchases above 21,400 are potential outliers.

The IQR method identified several extreme purchase values. Further investigation shows that these values represent valid transactions rather than data-entry errors. Therefore, the outliers were retained for analysis to avoid removing genuine high-value purchases.

Univariate Analysis

Univariate Analysis: Studying one column/variable at a time to understand it better.

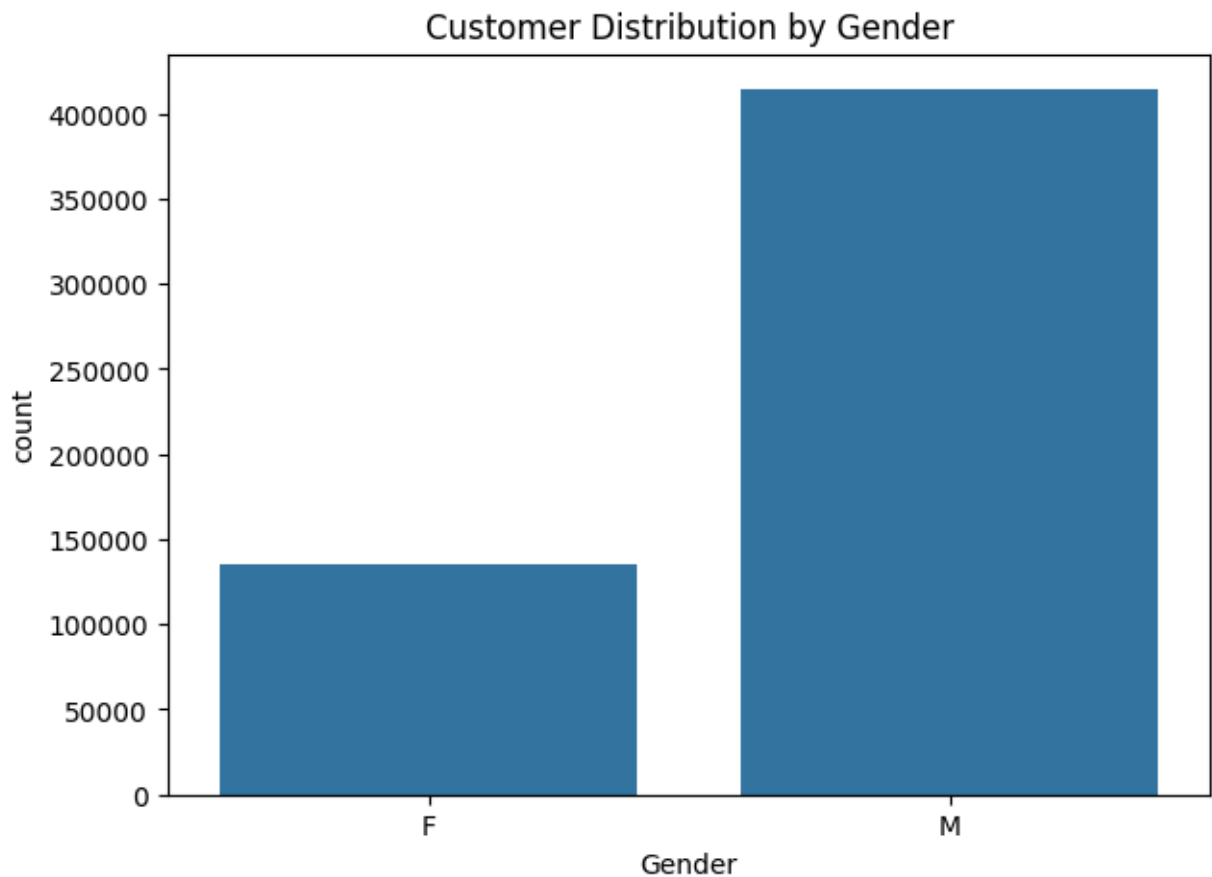
```

In [25]: # Gender
df["Gender"].value_counts(normalize=True).mul(100).round(2)

```

```
Out[25]: M    75.31  
        F    24.69  
        Name: Gender, dtype: float64
```

```
In [26]: # gender  
plt.figure(figsize=(7,5))  
  
sns.countplot(data=df, x="Gender")  
  
plt.title("Customer Distribution by Gender")  
plt.show()
```



Male customers are much higher than female customers.

The customer base is mostly male.

Female customers are much fewer.

```
In [27]: # Age  
df["Age"].value_counts()
```

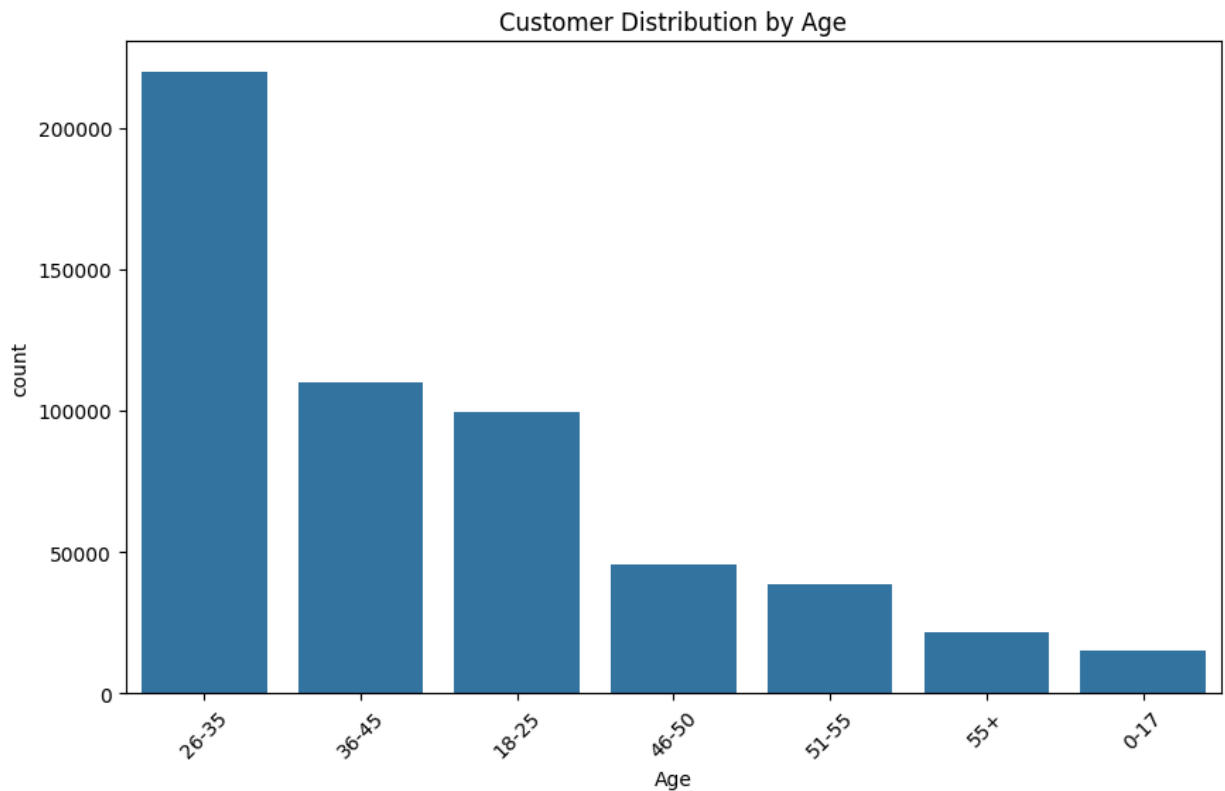
```
Out[27]: 26-35    219587  
        36-45    110013  
        18-25     99660  
        46-50     45701  
        51-55     38501  
        55+       21504  
        0-17      15102  
        Name: Age, dtype: int64
```

```
In [28]: # Age
plt.figure(figsize=(10,6))

sns.countplot(
    data=df,
    x="Age",
    order=df["Age"].value_counts().index
)

plt.title("Customer Distribution by Age")
plt.xticks(rotation=45)

plt.show()
```



26–35 age group has the highest number of customers.

36–45 is the second-highest group.

0–17 has the fewest customers.

Overall, customers are mainly in the 26–45 age groups.

```
In [29]: # City_Category
df["City_Category"].value_counts()
```

```
Out[29]: B    231173
         C    171175
         A    147720
         Name: City_Category, dtype: int64
```

```
In [30]: # city_Category
sns.countplot(data=df, x="City_Category")

plt.title("Customers by City")
plt.show()
```



City B has the highest number of customers.

City C has fewer customers than City B but more than City A.

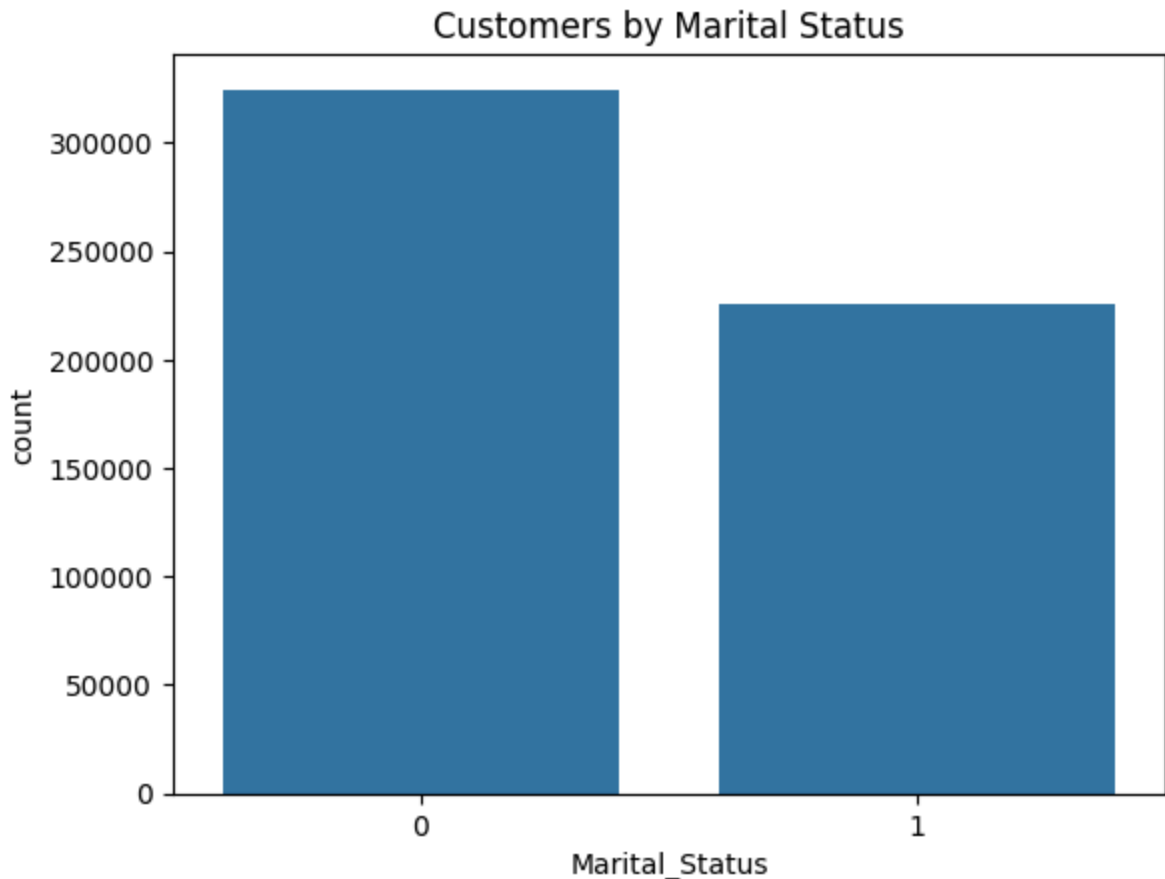
City A has the lowest number of customers.

```
In [31]: # Marital Status
df["Marital_Status"].value_counts()
```

```
Out[31]: 0    324731
         1    225337
         Name: Marital_Status, dtype: int64
```

```
In [32]: # Marital Status
sns.countplot(data=df, x="Marital_Status")

plt.title("Customers by Marital Status")
plt.show()
```



Marital status 0 has more customers than marital status 1.

The customer base is mostly from marital status 0.

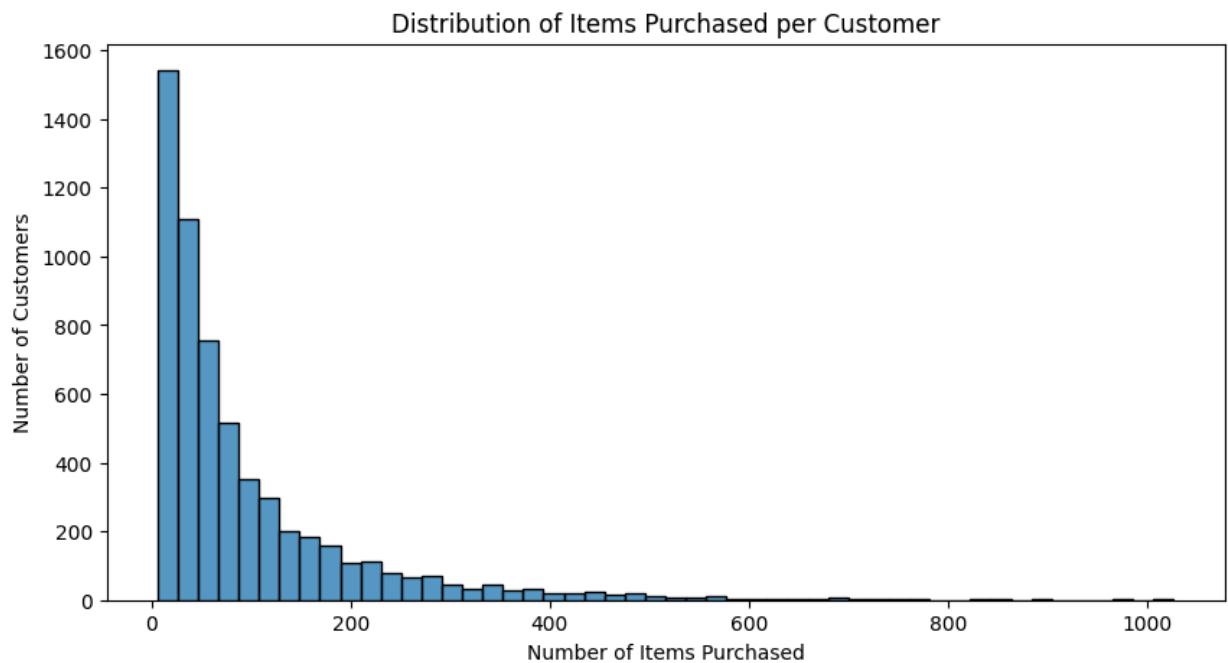
Customer Demographic Analysis

```
In [33]: customer_transactions = (
          df.groupby("User_ID").size().reset_index(name="Items_Purchased")
        )

print(f"Average items purchased per customer: {customer_transactions['Items_Purchased'].mean()}")
print(f"Median items purchased per customer: {customer_transactions['Items_Purchased'].median()}")
print(f"Min items: {customer_transactions['Items_Purchased'].min()}, Max items: {customer_transactions['Items_Purchased'].max()}")

# Distribution view – more useful than a single "repeat rate" number
plt.figure(figsize=(10, 5))
sns.histplot(customer_transactions["Items_Purchased"], bins=50)
plt.title("Distribution of Items Purchased per Customer")
plt.xlabel("Number of Items Purchased")
plt.ylabel("Number of Customers")
plt.show()
```

Average items purchased per customer: 93.4
Median items purchased per customer: 54
Min items: 6, Max items: 1026



This dataset records individual line items, not separate checkout sessions — so nearly every customer will appear multiple times regardless of loyalty.

A "repeat purchase rate" isn't meaningful here without an order ID or timestamp.

Instead, we look at items purchased per customer: average of 93.4, median of 54 — showing most customers bought a handful of items in this Black Friday period, with a long tail of high-volume shoppers.

```
In [34]: # Gender vs Average Purchase
gender_purchase = df.groupby("Gender")["Purchase"].agg(
    ["count", "mean", "sum"]
)

gender_purchase
```

```
Out[34]:
```

	count	mean	sum
Gender			
F	135809	8734.565765	1186232642
M	414259	9437.526040	3909580100

```
In [35]: # Age vs Purchase
age_purchase = df.groupby("Age")["Purchase"].agg(
    ["count", "mean", "sum"]
).sort_values("mean", ascending=False)

age_purchase
```

Out[35]:

	count	mean	sum
Age			
51-55	38501	9534.808031	367099644
55+	21504	9336.280459	200767375
36-45	110013	9331.350695	1026569884
26-35	219587	9252.690633	2031770578
46-50	45701	9208.625697	420843403
18-25	99660	9169.663606	913848675
0-17	15102	8933.464640	134913183

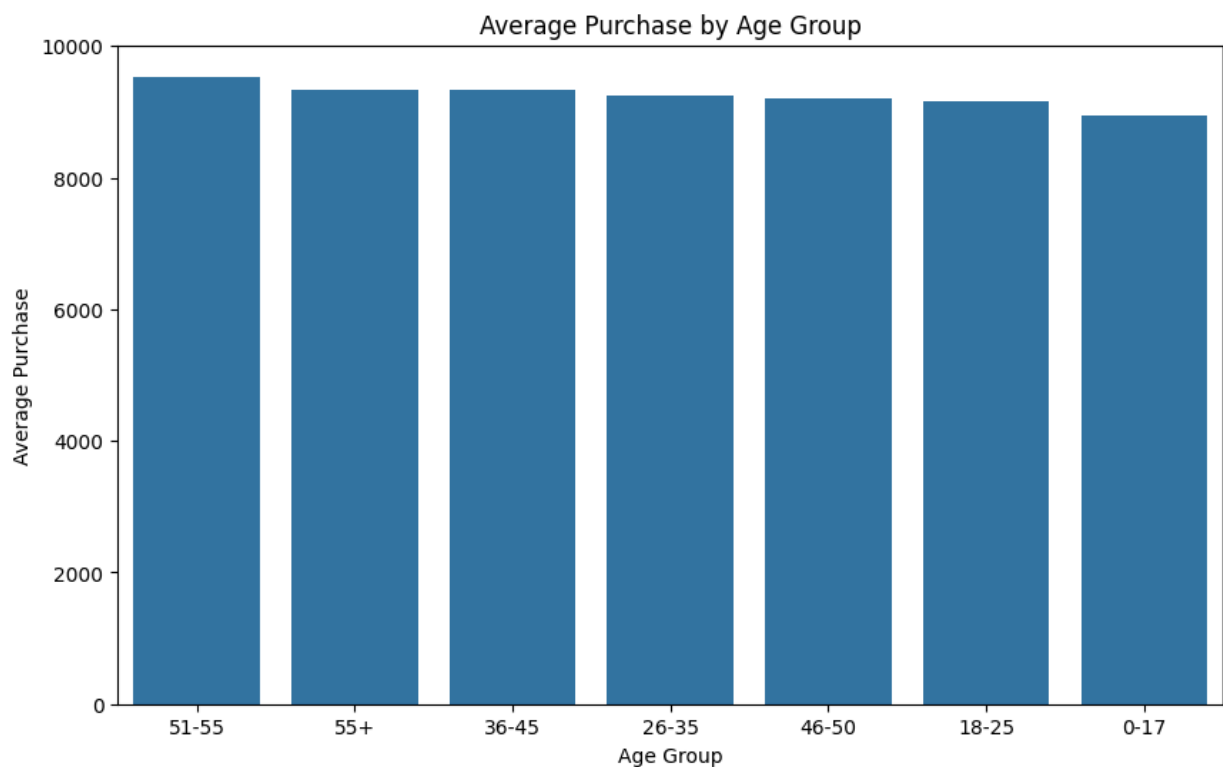
In [36]:

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=age_purchase.reset_index(),
    x="Age",
    y="mean"
)

plt.title("Average Purchase by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Average Purchase")

plt.show()
```



Customers aged 51–55 spend the most per purchase on average, though this group is smaller in size than the 26–35 group.

```
In [37]: # City vs Purchase
city_purchase = df.groupby("City_Category")["Purchase"].agg(
    ["count", "mean", "sum"]
)

city_purchase
```

```
Out[37]:
```

	count	mean	sum
City_Category			
A	147720	8911.939216	1316471661
B	231173	9151.300563	2115533605
C	171175	9719.920993	1663807476

```
In [38]: # Marital Status vs Purchase
marital_purchase = df.groupby("Marital_Status")["Purchase"].agg(
    ["count", "mean", "sum"]
)

marital_purchase
```

```
Out[38]:
```

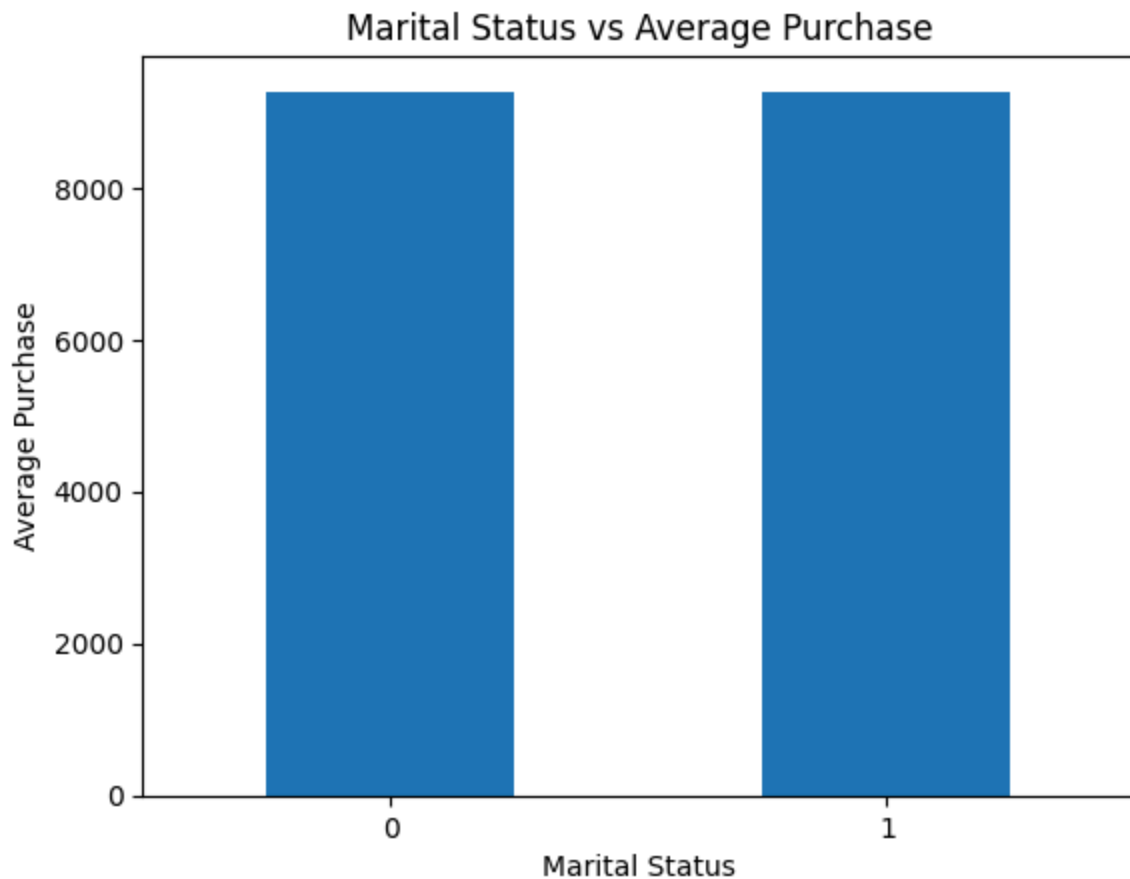
	count	mean	sum
Marital_Status			
0	324731	9265.907619	3008927447
1	225337	9261.174574	2086885295

```
In [39]: # Marital Status vs Average Purchase

marital_purchase = df.groupby("Marital_Status")["Purchase"].mean()

marital_purchase.plot(kind="bar")

plt.xlabel("Marital Status")
plt.ylabel("Average Purchase")
plt.title("Marital Status vs Average Purchase")
plt.xticks(rotation=0)
plt.show()
```



```
In [40]: # Gender + Marital Status
gender_marital = df.groupby(
    ["Gender", "Marital_Status"]
)["Purchase"].mean().reset_index()

gender_marital
```

```
Out[40]:
```

	Gender	Marital_Status	Purchase
0	F	0	8679.845815
1	F	1	8810.249789
2	M	0	9453.756740
3	M	1	9413.817605

```
In [41]: # Product Analysis
# product category 1
product1 = df.groupby("Product_Category_1")["Purchase"].agg(
    ["count", "mean", "sum"]
).sort_values("sum", ascending=False)

product1.head(10)
```

Out[41]:

		count	mean	sum
Product_Category_1				
1	140378	13606.218596	1910013754	
5	150933	6240.088178	941835229	
8	113925	7498.958078	854318799	
6	20466	15838.478550	324150302	
2	23864	11251.935384	268516186	
3	20213	10096.705734	204084713	
16	9828	14766.037037	145120612	
11	24287	4685.268456	113791115	
10	5125	19675.570927	100837301	
15	6290	14780.451828	92969042	

```
In [42]: # Top Product Categories by Revenue
top_categories = (
    df.groupby("Product_Category_1")["Purchase"]
      .sum()
      .sort_values(ascending=False)
      .head(10)
)

top_categories
```

```
Out[42]: Product_Category_1
1      1910013754
5      941835229
8      854318799
6      324150302
2      268516186
3      204084713
16     145120612
11     113791115
10     100837301
15      92969042
Name: Purchase, dtype: int64
```

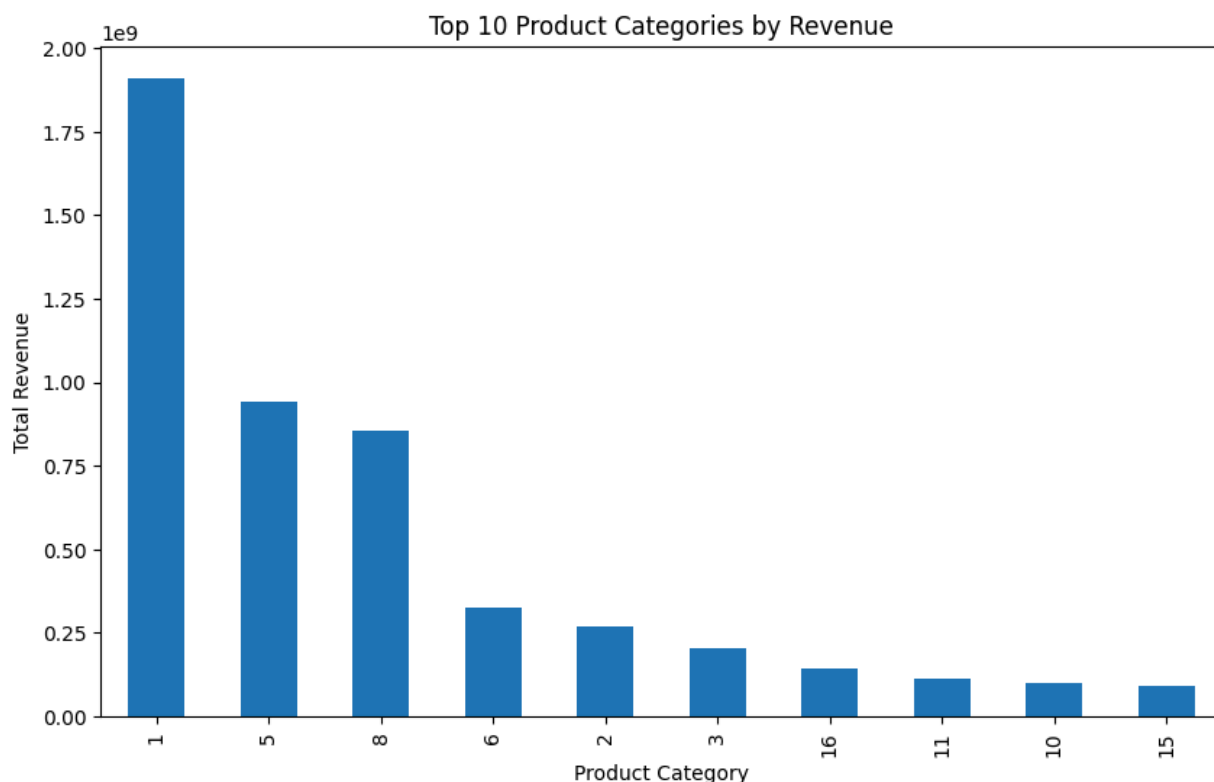
Top 10 Product Categories by Revenue

```
In [43]: plt.figure(figsize=(10,6))

top_categories.plot(kind="bar")

plt.title("Top 10 Product Categories by Revenue")
plt.xlabel("Product Category")
plt.ylabel("Total Revenue")

plt.show()
```



Category 1 is the top revenue contributor.

Categories 5 and 8 are the next strongest performers.

Revenue is concentrated in a few categories.

Lower-performing categories need further analysis to identify growth opportunities.

Recommendation: Focus on top categories while improving weaker ones.

Product Category by Average Purchase

```
In [44]: # Top 10 product categories by average purchase
category_avg = (
    df.groupby("Product_Category_1")["Purchase"]
      .mean()
      .sort_values(ascending=False)
      .head(10)
)

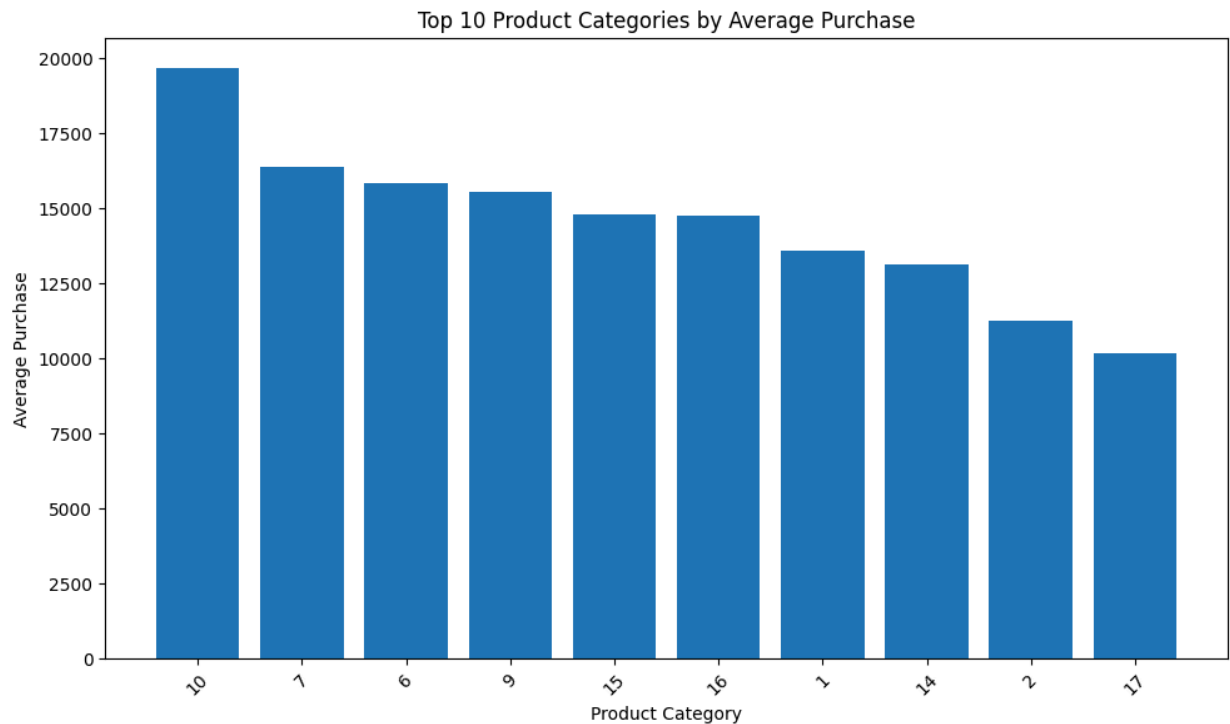
# Bar graph
plt.figure(figsize=(10, 6))

plt.bar(category_avg.index.astype(str), category_avg.values)

plt.xlabel("Product Category")
plt.ylabel("Average Purchase")
plt.title("Top 10 Product Categories by Average Purchase")

plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```



Category 10 has the highest average purchase value.

Categories 7, 6, and 9 also show strong customer spending.

Categories 2 and 17 have the lowest average purchase value.

Recommendation: Focus on high-value categories and improve the average purchase value of weaker categories.

Customer-Level Analysis

```
In [45]: customer_clv = (
    df.groupby("User_ID")
    .agg(Total_Purchase=("Purchase", "sum"), Number_of_Transactions=("Purchase", "count"))
    .reset_index()
)

# Avg order value - spend per line item, NOT lifetime value
customer_clv["Avg_Order_Value"] = customer_clv["Total_Purchase"] / customer_clv["Number_of_Transactions"]

customer_clv.head()
```

Out[45]:

	User_ID	Total_Purchase	Number_of_Transactions	Avg_Order_Value
0	1000001	334093	35	9545.514286
1	1000002	810472	77	10525.610390
2	1000003	341635	29	11780.517241
3	1000004	206468	14	14747.714286
4	1000005	821001	106	7745.292453

This dataset has no transaction date, so a projected CLV can't be calculated.

Total_Purchase is used here as a CLV proxy (cumulative spend to date).

Avg_Order_Value is a separate metric — average spend per line item, not lifetime value.

```
In [46]: category_cols = [
            "Product_Category_1",
            "Product_Category_2",
            "Product_Category_3"
        ]

        category_breadth = (
            df.groupby("User_ID")[category_cols]
              .apply(lambda x: x.stack().nunique())
              .reset_index(name="Category_Breadth")
        )

        category_breadth.head()
```

Out[46]:

	User_ID	Category_Breadth
0	1000001	15
1	1000002	16
2	1000003	12
3	1000004	12
4	1000005	17

```
In [47]: df["Stay_Years"] = (
            df["Stay_In_Current_City_Years"]
              .replace("4+", 4)
              .astype(int)
        )
```

```
In [48]: city_loyalty = (
            df.groupby("City_Category")
              .agg(
                  Avg_Purchase=("Purchase", "mean"),
                  Stay_Years=("Stay_Years", "mean")
              )
              .reset_index()
```

```
)

city_loyalty["City_Loyalty_Index"] = (
    city_loyalty["Avg_Purchase"] *
    city_loyalty["Stay_Years"]
)

city_loyalty = city_loyalty.round(2)

city_loyalty
```

Out[48]:

	City_Category	Avg_Purchase	Stay_Years	City_Loyalty_Index
0	A	8911.94	1.81	16121.43
1	B	9151.30	1.88	17159.51
2	C	9719.92	1.88	18259.60

In [49]:

```
customer_analysis = (
    df.groupby("User_ID")
    .agg(
        Total_Purchase=("Purchase", "sum"),
        Transactions=("Purchase", "count")
    )
    .reset_index()
)

customer_analysis.head()
```

Out[49]:

	User_ID	Total_Purchase	Transactions
0	1000001	334093	35
1	1000002	810472	77
2	1000003	341635	29
3	1000004	206468	14
4	1000005	821001	106

In [50]:

```
# Top 10 Customers
top_customers = customer_analysis.sort_values(
    "Total_Purchase",
    ascending=False
).head(10)

top_customers
```

Out[50]:

	User_ID	Total_Purchase	Transactions
4166	1004277	10536909	979
1634	1001680	8699596	1026
2831	1002909	7577756	718
1885	1001941	6817493	898
416	1000424	6573609	694
4335	1004448	6566245	575
5683	1005831	6512433	727
981	1001015	6511314	740
3297	1003391	6477160	698
1142	1001181	6387961	862

Top 10 Customers by Total Purchase

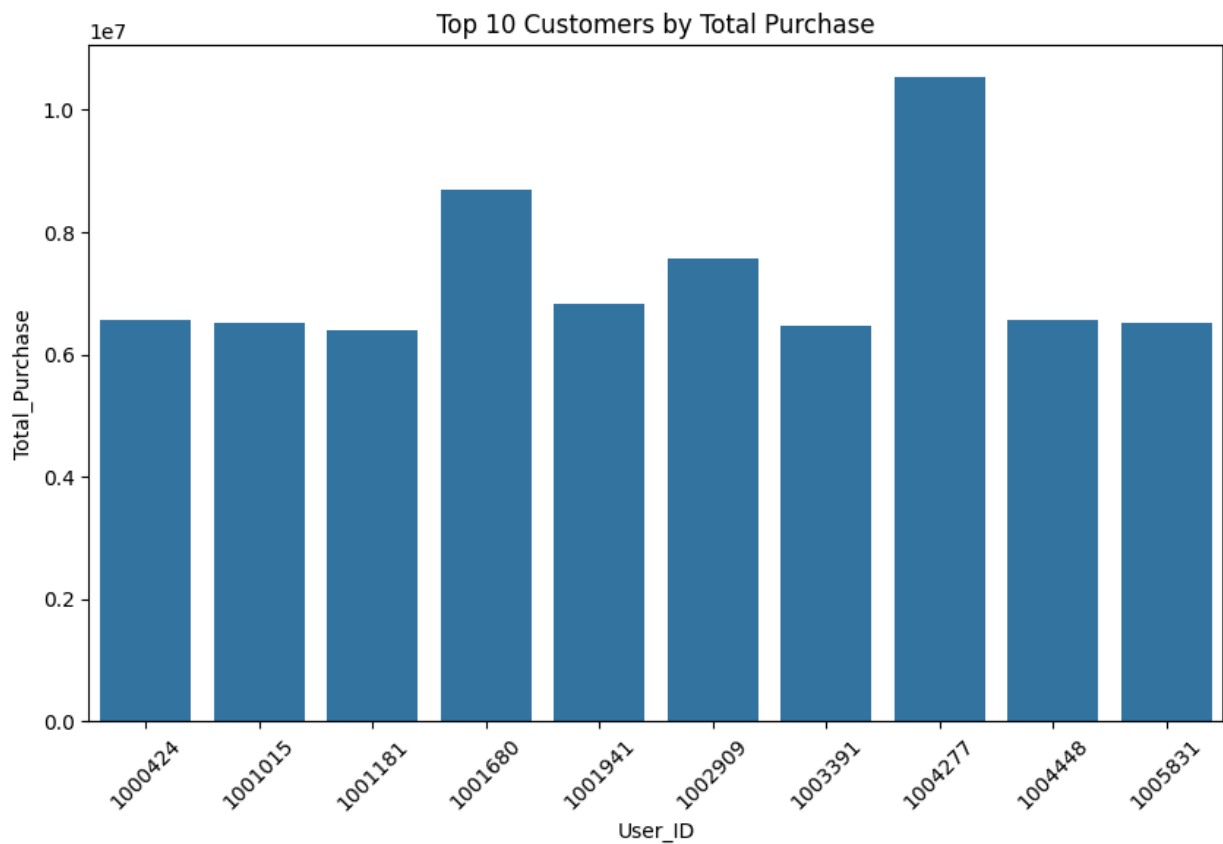
```
In [51]: plt.figure(figsize=(10,6))

sns.barplot(
    data=top_customers,
    x="User_ID",
    y="Total_Purchase"
)

plt.title("Top 10 Customers by Total Purchase")

plt.xticks(rotation=45)

plt.show()
```



Customer 1004277 has the highest total purchase, while the other customers have relatively similar purchase values.

```
In [52]: # Customer Segmentation
customer_analysis["Customer_Segment"] = pd.qcut(
    customer_analysis["Total_Purchase"],
    q=3,
    labels=["Low Value", "Medium Value", "High Value"]
)
```

```
In [53]: customer_analysis["Customer_Segment"].value_counts()
```

```
Out[53]: Low Value      1964
High Value      1964
Medium Value    1963
Name: Customer_Segment, dtype: int64
```

```
In [54]: customer_analysis.groupby(
    "Customer_Segment"
)["Total_Purchase"].agg(
    ["count", "mean", "sum"]
)
```

Out[54]:

		count	mean	sum
Customer_Segment				
	Low Value	1964	1.879863e+05	369205122
	Medium Value	1963	5.425764e+05	1065077527
	High Value	1964	1.864323e+06	3661530093

Cross-Selling Analysis

It identifies which products are often purchased together.

This helps businesses recommend related products and increase sales.

In [55]:

```
# Cross-Selling Analysis

valid_df = df[df["Product_Category_2"] != "Unknown"]

cross_sell = (
    valid_df
    .groupby(["Product_Category_1", "Product_Category_2"])
    .size()
    .reset_index(name="Number_of_Purchases")
    .sort_values("Number_of_Purchases", ascending=False)
    .reset_index(drop=True)
)

cross_sell["Purchase_Percentage"] = (
    cross_sell["Number_of_Purchases"] / len(valid_df) * 100
).round(2)

cross_sell.head(10)
```

Out[55]:

	Product_Category_1	Product_Category_2	Number_of_Purchases	Purchase_Percentage
0	1	2	49217	13.07
1	5	8	31670	8.41
2	5	14	25588	6.80
3	8	14	20771	5.52
4	1	15	18544	4.93
5	3	4	16858	4.48
6	1	8	15471	4.11
7	1	16	14828	3.94
8	6	8	12071	3.21
9	1	5	11267	2.99

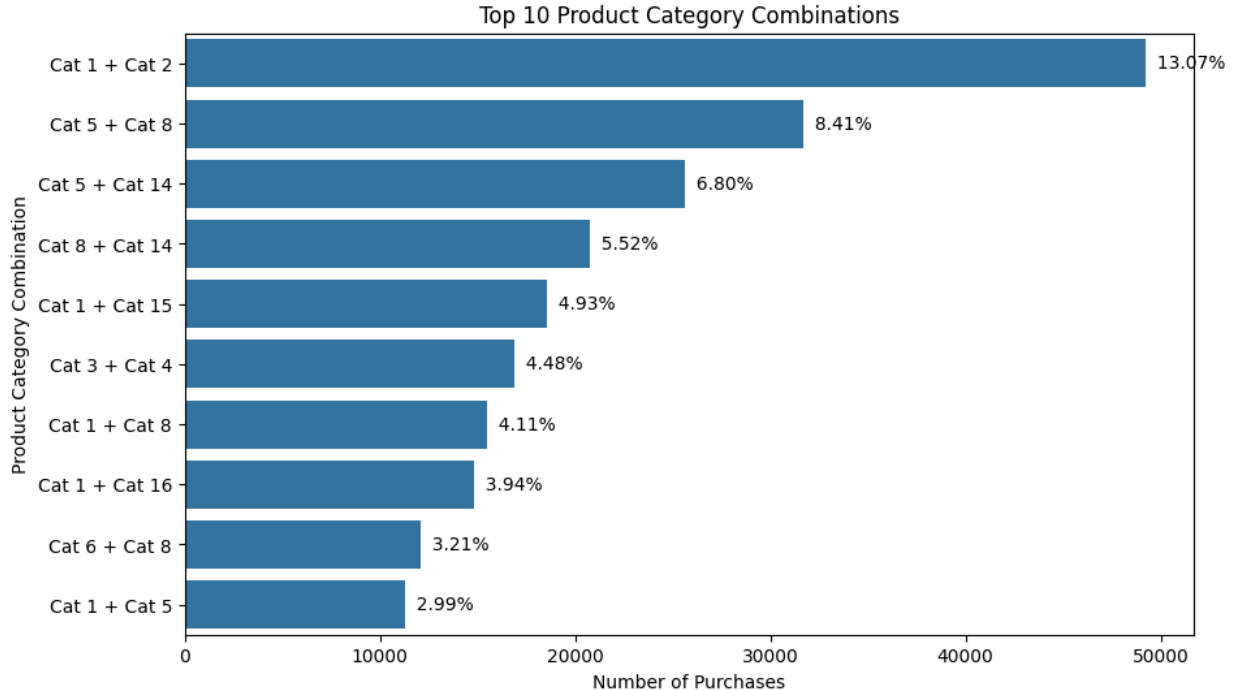
```
In [56]: # Build top_cross_sell: top 10 category combinations with a readable Label
top_cross_sell = cross_sell.head(10).copy()
top_cross_sell["Combination"] = (
    "Cat " + top_cross_sell["Product_Category_1"] + " + Cat " + top_cross_sell["Product_Category_2"]
)
```

```
In [57]: plt.figure(figsize=(10, 6))

ax = sns.barplot(
    data=top_cross_sell,
    x="Number_of_Purchases",
    y="Combination",
    order=top_cross_sell["Combination"]
)
for i, row in top_cross_sell.reset_index(drop=True).iterrows():
    ax.text(
        row["Number_of_Purchases"],
        i,
        f' {row["Purchase_Percentage"]:.2f}%',
        va="center"
    )

plt.title("Top 10 Product Category Combinations")
plt.xlabel("Number of Purchases")
plt.ylabel("Product Category Combination")

plt.show()
```



"Product category 1 + 2 is the most common combination, accounting for 13.07% of purchases.

This shows customers frequently buy these two categories together, making them a good combination for cross-selling and bundle offers."

Product Category 3 contains a high proportion of missing values (approximately 70%). Therefore, it is retained for completeness but is not prioritized in cross-selling analysis. Product Category 1 and Product Category 2 provide more reliable coverage for identifying common product combinations.

Bivariate Analysis

"Bivariate analysis helps us understand the relationship between two variables and identify patterns or differences between them."

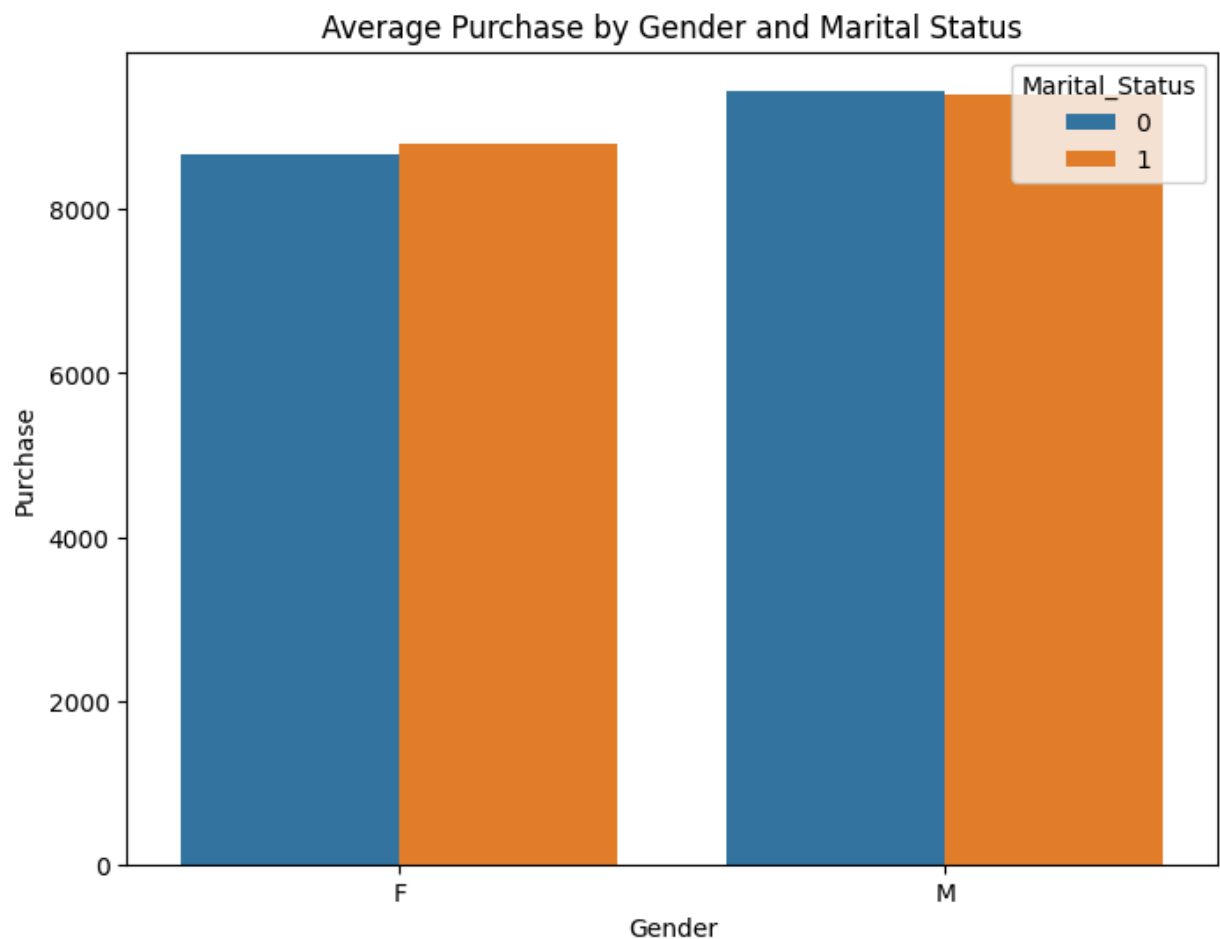
Average Purchase by Gender and Marital Status

```
In [58]: plt.figure(figsize=(8,6))

sns.barplot(
    data=gender_marital,
    x="Gender",
    y="Purchase",
    hue="Marital_Status"
)

plt.title("Average Purchase by Gender and Marital Status")

plt.show()
```



“Male customers have slightly higher average purchases, while marital status shows little difference.”

```
In [59]: # Age + City Analysis
age_city = df.groupby(
    ["Age", "City_Category"]
)["Purchase"].mean().reset_index()

age_city
```

Out[59]:

	Age	City_Category	Purchase
0	0-17	A	8615.110456
1	0-17	B	8917.295308
2	0-17	C	9059.503299
3	18-25	A	8833.734084
4	18-25	B	9031.706985
5	18-25	C	9696.570919
6	26-35	A	8952.503004
7	26-35	B	9149.193178
8	26-35	C	9835.388993
9	36-45	A	8990.333997
10	36-45	B	9107.901067
11	36-45	C	9882.012654
12	46-50	A	8348.526752
13	46-50	B	9247.927129
14	46-50	C	9533.184023
15	51-55	A	9508.505001
16	51-55	B	9340.911392
17	51-55	C	9780.380806
18	55+	A	8485.945424
19	55+	B	9803.560635
20	55+	C	9385.316939

Average Purchase by Age and City

```
In [60]: plt.figure(figsize=(12,6))

sns.barplot(
    data=age_city,
    x="Age",
    y="Purchase",
```

```

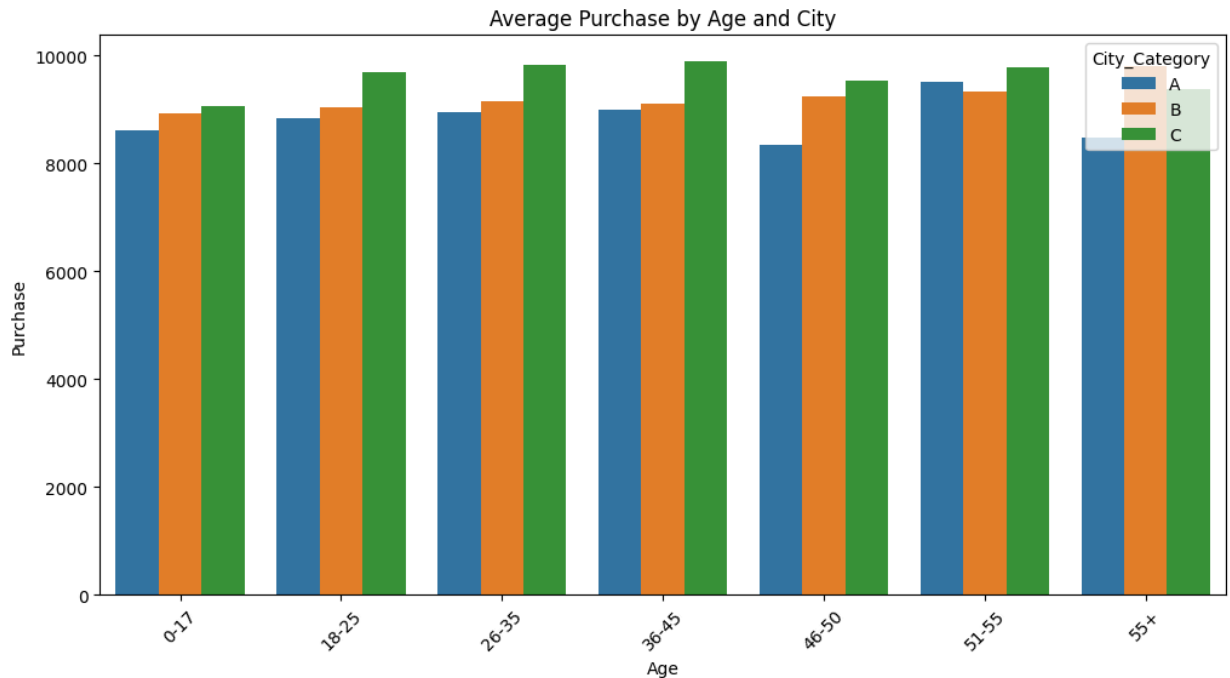
hue="City_Category"
)

plt.title("Average Purchase by Age and City")

plt.xticks(rotation=45)

plt.show()

```



"City C has the highest average purchase across most age groups, while City A is generally lower."

Correlation Analysis "Correlation analysis shows how strongly two numerical variables are related."

```

In [61]: # Encode Age ordinally so it can be used in correlation
age_order = {
    "0-17": 0, "18-25": 1, "26-35": 2, "36-45": 3,
    "46-50": 4, "51-55": 5, "55+": 6
}
df["Age_Ordinal"] = df["Age"].map(age_order)

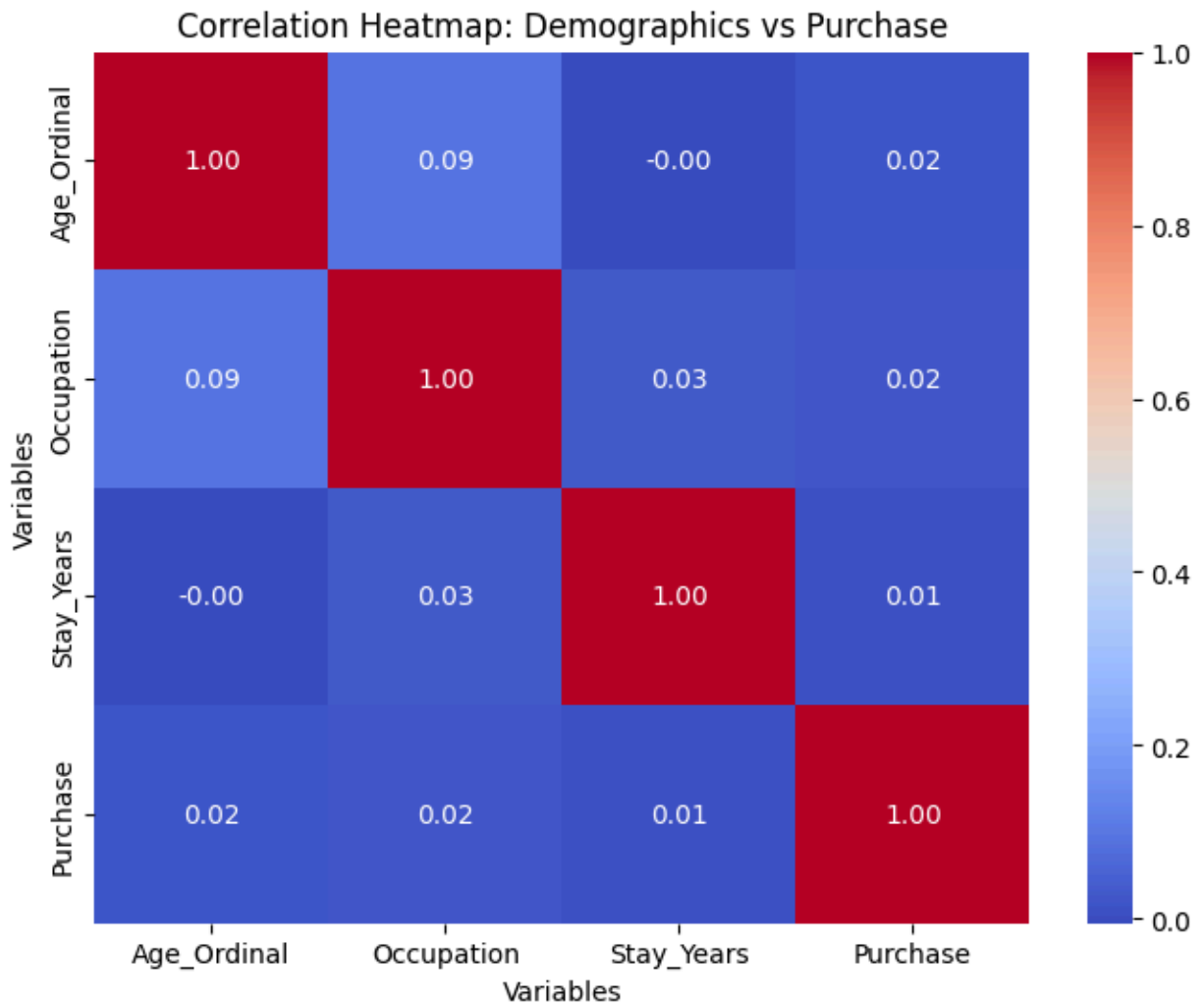
# Select meaningful numerical variables
corr_data = df[["Age_Ordinal", "Occupation", "Stay_Years", "Purchase"]]

# Calculate correlation
correlation_matrix = corr_data.corr()

# Plot correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap: Demographics vs Purchase")
plt.xlabel("Variables")

```

```
plt.ylabel("Variables")
plt.show()
```



None of the demographic variables show a meaningful linear relationship with Purchase:

Age_Ordinal (0.02), Occupation (0.02), and Stay_Years (0.01) are all effectively zero.

Age and Occupation show a weak relationship with each other (0.09), but neither correlates with spend.

None of the demographic variables show a meaningful linear relationship with Purchase:

Age_Ordinal (0.02), Occupation (0.02), and Stay_Years (0.01) are all effectively zero.

Age and Occupation show a weak relationship with each other (0.09), but neither correlates with spend.

```
In [62]: age_purchase = df.groupby("Age")["Purchase"].mean().reset_index()

age_purchase
```

Out[62]:

	Age	Purchase
0	0-17	8933.464640
1	18-25	9169.663606
2	26-35	9252.690633
3	36-45	9331.350695
4	46-50	9208.625697
5	51-55	9534.808031
6	55+	9336.280459

Average Purchase by Age Group

In [63]:

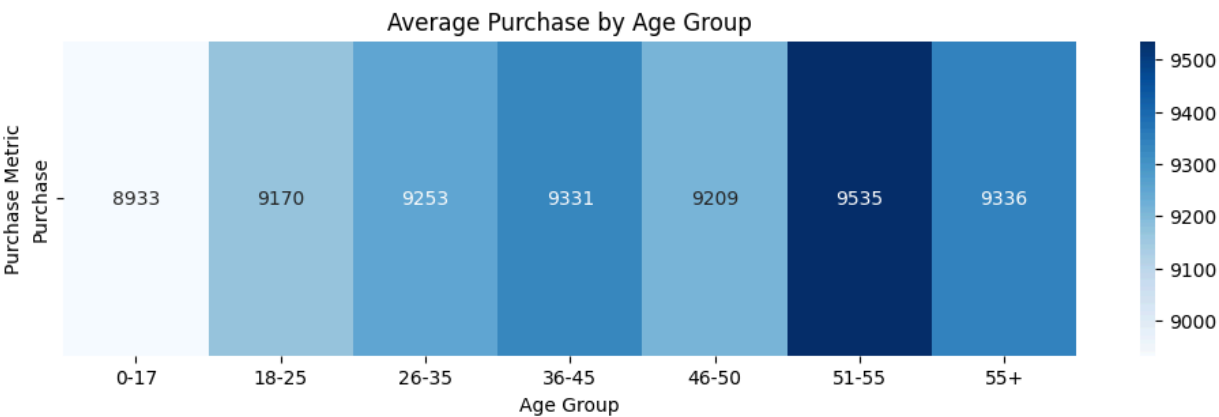
```
age_purchase_matrix = age_purchase.set_index("Age").T

plt.figure(figsize=(12, 3))

sns.heatmap(
    age_purchase_matrix,
    annot=True,
    fmt=".0f",
    cmap="Blues"
)

plt.title("Average Purchase by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Purchase Metric")

plt.show()
```



“Purchase is highest for the 51–55 age group and lowest for the 0–17 age group.”

Average Purchase by Age Group and Gender

In [64]:

```
demo_heatmap = pd.pivot_table(
    df,
    values="Purchase",
    index="Age",
    columns="Gender",
```

```

aggfunc="mean"
)

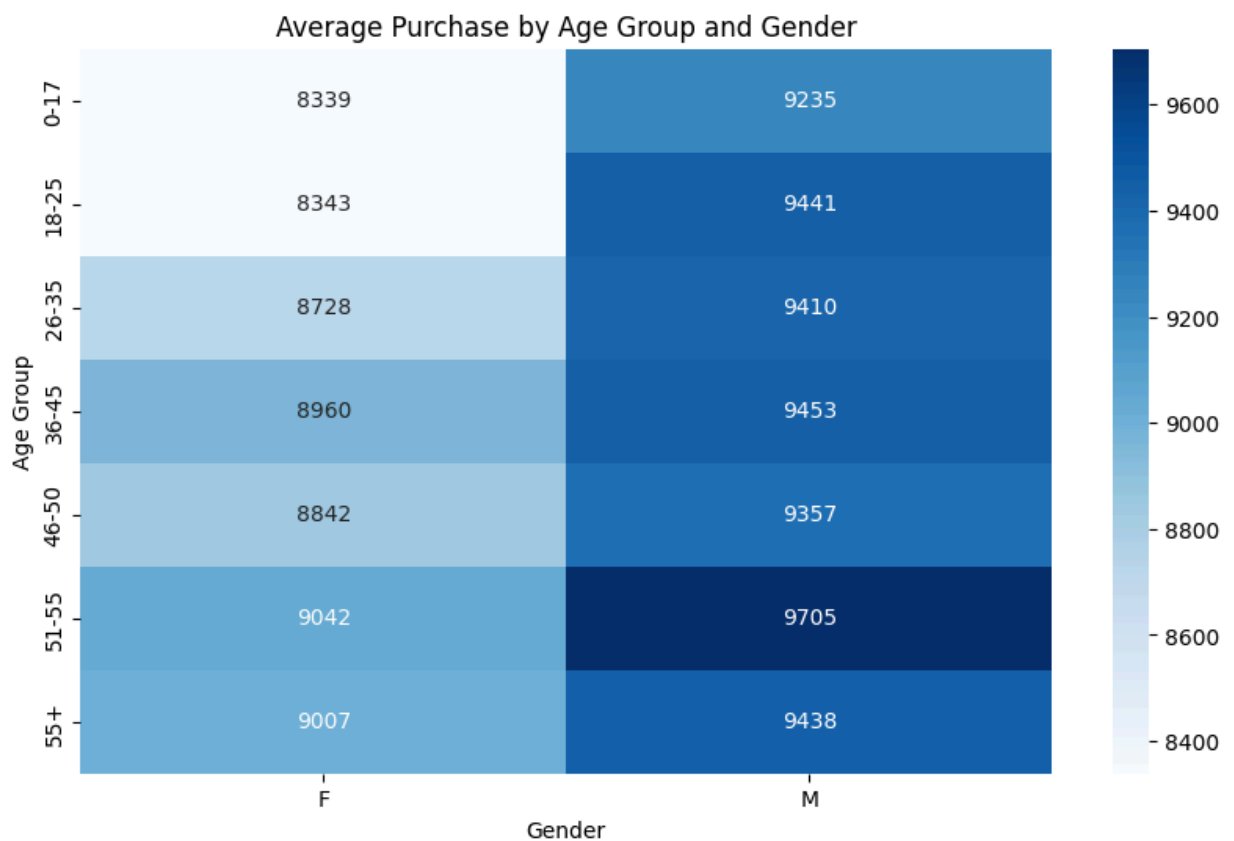
plt.figure(figsize=(10, 6))

sns.heatmap(
    demo_heatmap,
    annot=True,
    fmt=".0f",
    cmap="Blues"
)

plt.title("Average Purchase by Age Group and Gender")
plt.xlabel("Gender")
plt.ylabel("Age Group")

plt.show()

```



"Male customers have higher average purchases than females in every age group."

Average Purchase by Age Group and City

```

In [65]: city_age_heatmap = pd.pivot_table(
    df,
    values="Purchase",
    index="Age",
    columns="City_Category",
    aggfunc="mean"
)

plt.figure(figsize=(10, 6))

```

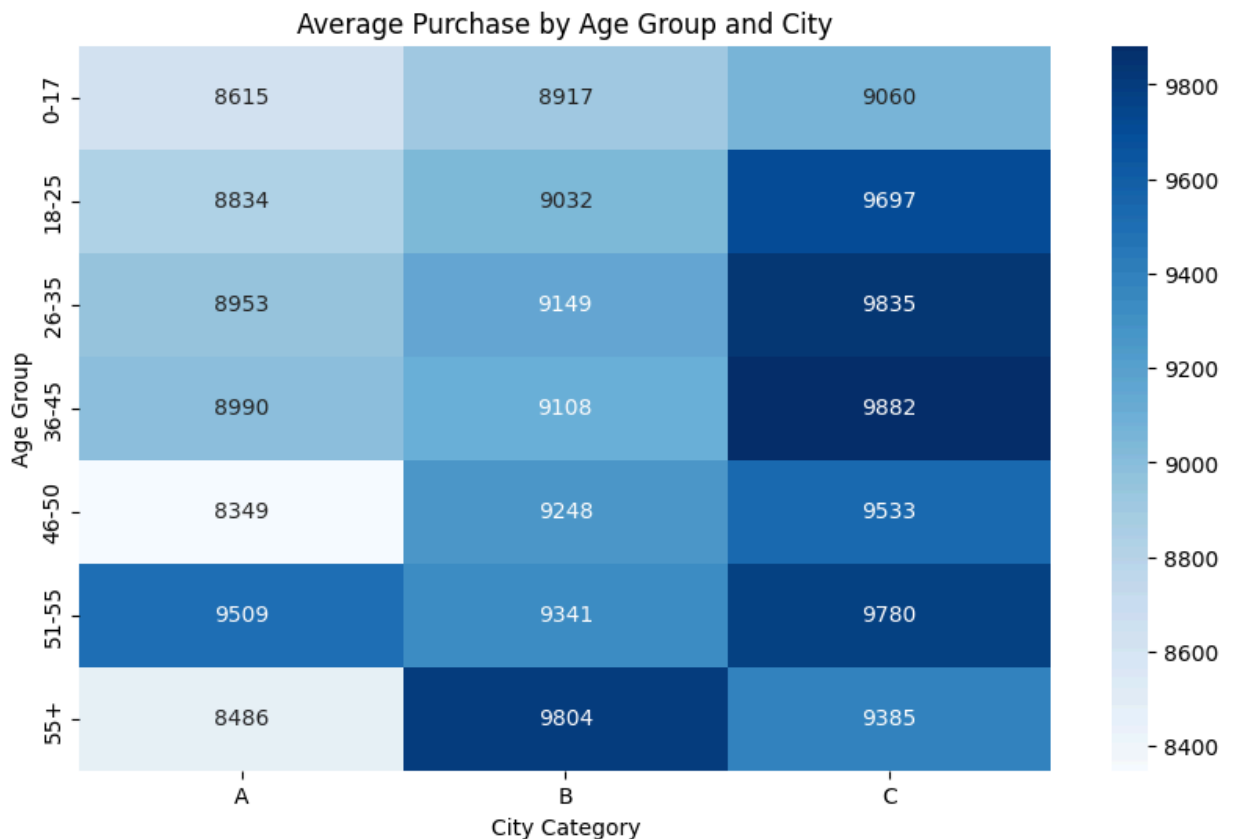
```

sns.heatmap(
    city_age_heatmap,
    annot=True,
    fmt=".0f",
    cmap="Blues"
)

plt.title("Average Purchase by Age Group and City")
plt.xlabel("City Category")
plt.ylabel("Age Group")

plt.show()

```



"City C has the highest average purchase across most age groups, while City A is generally lower."

Statistical Hypothesis Testing

"Statistical hypothesis testing helps us determine whether the observed differences in purchase behavior are statistically significant."

Gender H0

There is no significant difference in purchase between males and females.

H1

There is a significant difference.

```
In [66]: from scipy.stats import ttest_ind

# Purchase values for male customers
male = df[df["Gender"] == "M"]["Purchase"]

# Purchase values for female customers
female = df[df["Gender"] == "F"]["Purchase"]

# Independent t-test
t_stat, p_value = ttest_ind(male, female)

print("t_stat:", round(t_stat, 2))
print("p_value:", p_value)

# Decision
if p_value < 0.05:
    print("Reject H0: There is a significant difference in average purchase between male and female customers.")
else:
    print("Fail to reject H0: There is no significant difference in average purchase between male and female customers.")
```

t_stat: 44.84
p_value: 0.0
Reject H0: There is a significant difference in average purchase between male and female customers.

"Since the p-value is 0.0 (< 0.05), we reject H₀. There is a significant difference in average purchase between male and female customers."

Marital Status Hypothesis Test

```
In [67]: single=df[df["Marital_Status"]==0]["Purchase"]
married=df[df["Marital_Status"]==1]["Purchase"]

t_stat,p_value=ttest_ind(single,married)

print("t_stat:",round((t_stat),2))
print("p_value:",round((p_value),2))

if p_value < 0.05:
    print("Reject H0")
    print("There is a significant difference in average purchase between single and married customers.")
else:
    print("Fail to Reject H0")
    print("There is no significant difference in average purchase between single and married customers.")
```

t_stat: 0.34
p_value: 0.73
Fail to Reject H0
There is no significant difference in average purchase between single and married customers

"Since the p-value is 0.73 (> 0.05), we fail to reject H_0 . There is no significant difference in average purchase between single and married customers."

ANOVA — City

```
In [68]: city_A= df[df["City_Category"]=="A"]["Purchase"]
city_B= df[df["City_Category"]=="B"]["Purchase"]
city_c= df[df["City_Category"]=="C"]["Purchase"]

f_stat,p_value=stats.f_oneway(city_A,city_B,city_c)

print("f_stat:",f_stat)
print("p_value:",p_value)

if p_value < 0.05:
    print("Reject H0")
    print("There is a significant difference in average purchase among the city categories.")
else:
    print("Fail to Reject H0")
    print("There is no significant difference in average purchase among the city categories.")

f_stat: 1130.7459610102974
p_value: 0.0
Reject H0
There is a significant difference in average purchase among the city categories.
```

"Since the p-value is 0.0 (< 0.05), we reject H_0 . There is a significant difference in average purchase among the three city categories."

ANOVA — Age Groups

```
In [69]: age_groups = [
    df[df["Age"] == age]["Purchase"]
    for age in df["Age"].unique()
]

f_stat, p_value = stats.f_oneway(*age_groups)

print("F-statistic:", f_stat)
print("P-value:", p_value)
```

```
F-statistic: 40.57579909450408
P-value: 1.0535639392516708e-49
```

"Since the p-value is very small (< 0.05), we reject H_0 . Average purchase differs significantly across age groups."

Final Business Insights

26–35 age group is the largest customer segment.

City B has the highest number of transactions.

Male customers dominate transactions and purchase value.

Some product categories generate higher revenue than others.

High-value customers contribute significantly to total revenue.

Gender has a significant impact on purchase behavior.

Marital status has no significant impact on average purchase.

Some product categories are frequently purchased together, creating cross-selling opportunities.

Business Recommendations

- Target the 26–35 age group with personalized offers and campaigns, as they form the largest customer segment.
- Focus marketing efforts on City B to strengthen sales in the highest-transaction market.
- Target male customers with relevant promotions, while also creating campaigns to increase female customer participation.
- Promote high-revenue product categories through discounts, featured placements, and targeted campaigns.
- Retain high-value customers using loyalty programs, exclusive offers, and personalized recommendations.
- Use gender-based purchase insights to create more targeted marketing strategies, as purchase behavior differs significantly by gender.
- Avoid heavily segmenting offers by marital status, since it does not significantly affect average purchase value.
- Create bundle offers for frequently purchased product combinations to increase cross-selling and average order value.

Overall Recommendation

“Focus on high-performing customer segments, cities, and products while using personalized promotions and cross-selling strategies to increase sales and customer value.”

Recommendation 1 — Target high-value customers

Create loyalty offers for customers in the High Value segment.

Recommendation 2 — Gender-based campaigns

Since your hypothesis test shows a significant gender difference, consider differentiated campaigns.

Recommendation 3 — Focus on high-performing products

Promote products/categories generating the highest revenue.

Recommendation 4 — Cross-selling

Bundle frequently purchased product categories.

Recommendation 5 — City-specific marketing

Use city-level purchase behavior to create targeted campaigns.

```
In [70]: df.to_csv("cleaned__black.csv", index=False)
```

```
In [71]: import pandas as pd
        from sqlalchemy import create_engine
```

```
In [72]: df = pd.read_csv("cleaned__black.csv")
```

```
In [73]: df
```

Out[73]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Mi
0	1000001	P00069042	F	0-17	10	A	2	
1	1000001	P00248942	F	0-17	10	A	2	
2	1000001	P00087842	F	0-17	10	A	2	
3	1000001	P00085442	F	0-17	10	A	2	
4	1000002	P00285442	M	55+	16	C	4+	
...	...	...	...	...	...	...	...	...
550063	1006033	P00372445	M	51-55	13	B	1	
550064	1006035	P00375436	F	26-35	1	C	3	
550065	1006036	P00375436	F	26-35	15	B	4+	
550066	1006038	P00375436	F	55+	1	C	2	
550067	1006039	P00371644	F	46-50	0	B	4+	

550068 rows × 14 columns

◀

▶

In [74]:

engine = create_engine("mysql+pymysql://root:1234@localhost:3306/blackfriday")

In [75]:

df.to_sql(
 name="sales",
 con=engine,
 if_exists="replace",
 index=False
)

Out[75]:

550068

In [76]:

print(df.shape)
df.head()

(550068, 14)

Out[76]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status
0	1000001	P00069042	F	0-17	10	A	2	
1	1000001	P00248942	F	0-17	10	A	2	
2	1000001	P00087842	F	0-17	10	A	2	
3	1000001	P00085442	F	0-17	10	A	2	
4	1000002	P00285442	M	55+	16	C	4+	

